Fundusze Europejskie Wiedza Edukacja Rozwój
Politechnika Wrocławska
Unia Europejska Europejski Fundusz Społeczny

„ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej”

Projektowanie gier dla różnych platform.

Marek Kopel

https://pwr-edu.zoom.us/rec/share/3WQUo6JxHYsZfntcmuQHPdTQ-OIVx2VXJ8r9o1NeAX8OsrZrLWWgdzuh78Nw9WvK.05PkwVJkV8QJQZPZ

Lecture

1. **Silniki gier. Wprowadzenie do Unity. Pierwsza gra 2D.**
2. **Mechanika 2D, animacje.**
3. **Oświetlenie, tekstury, materiały. Pierwsza gra 3D.**
4. Narzędzia wspierające, np. Blender.
5. **Zapis/odczyt danych, komunikacja sieciowa.**
6. **Modelowanie terenu. Generowanie terenu.**
7. **Sztuczna inteligencja w grach.**
8. Virtual Reality, wsparcie VR w Unity.
9. **Optymalizacje w Unity.**
10. Historia i klasyfikacja gier.
11. Prototypowanie gier. GDD.
12. Projektowanie poziomów gier.
13. Projektowanie gier dla różnych platform.
14. Testowanie gier. Test.
15. Test.

Cross-Platform Game Engines

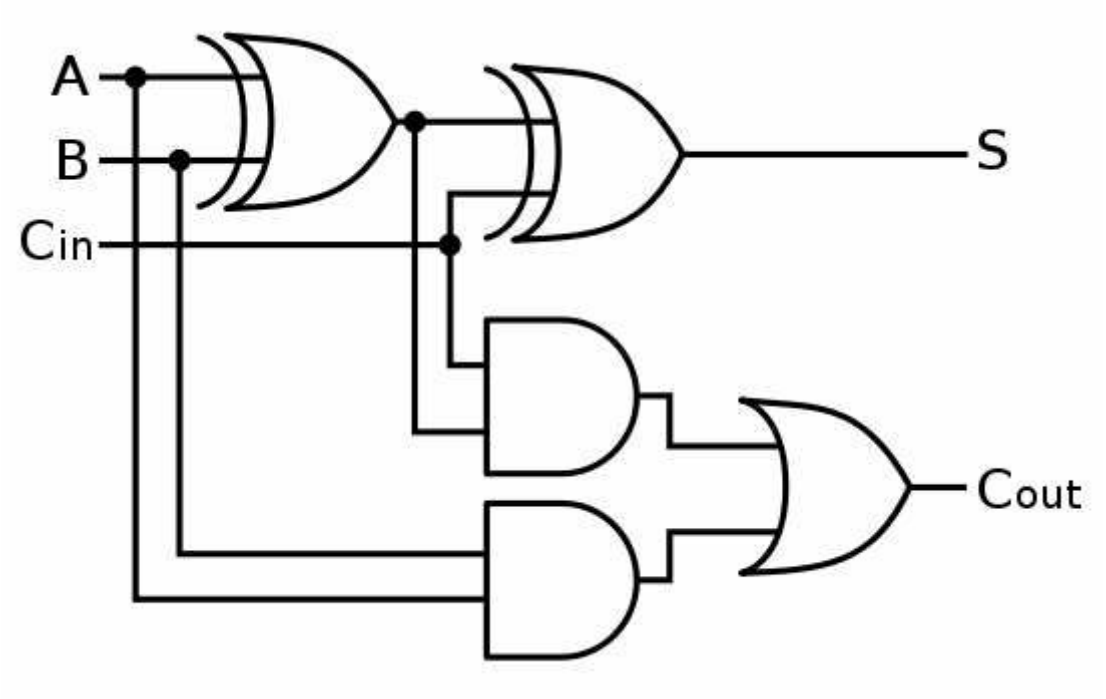
based on https://www.slideshare.net/kiyoungmoon144/crossplatform-game-engine

Computer game is a computer software

- It’s important!
  - Computer Games = Computer Software
- To make a computer software we should understand about the computer.

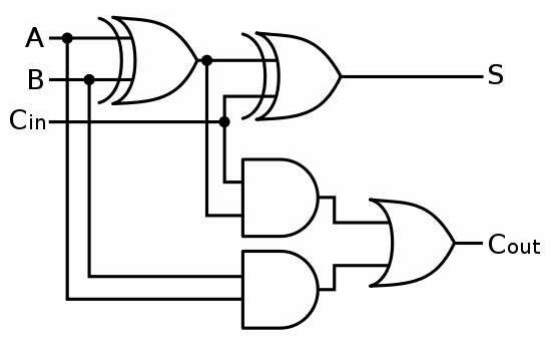
Full adder

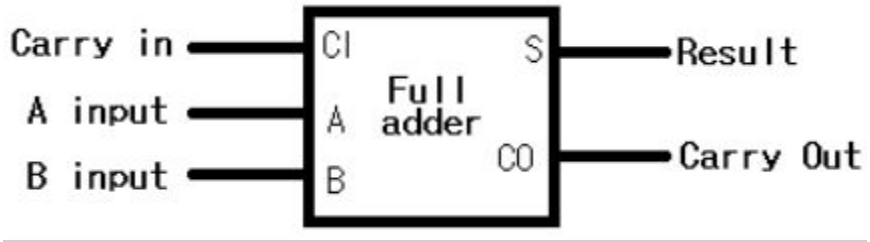
- Take 3 inputs and 2 outputs.



Let’s compare these.

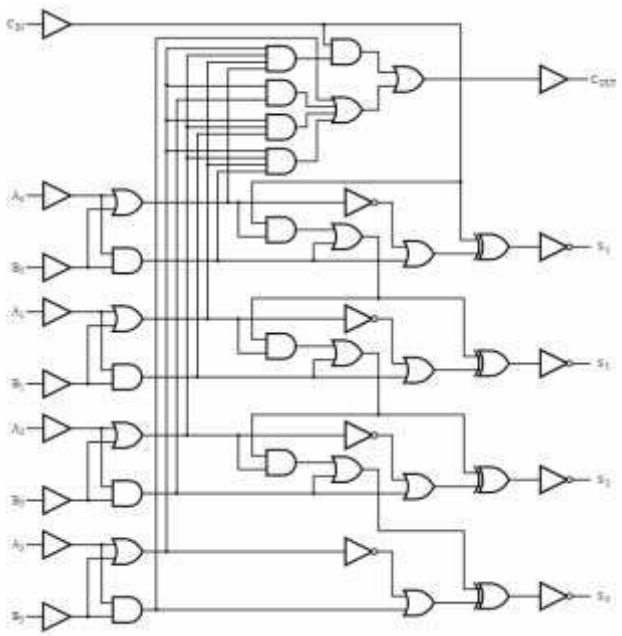
- Easy to use, remember.





4 bits adder is complicated

- Let’s use abstraction!



Instruction Set

- x86, i4004
- We can use Computer language to make a computer software.

[Those instructions preceded by an asterisk (*) are 2 word instructions that occupy 2 successive locations in ROM]

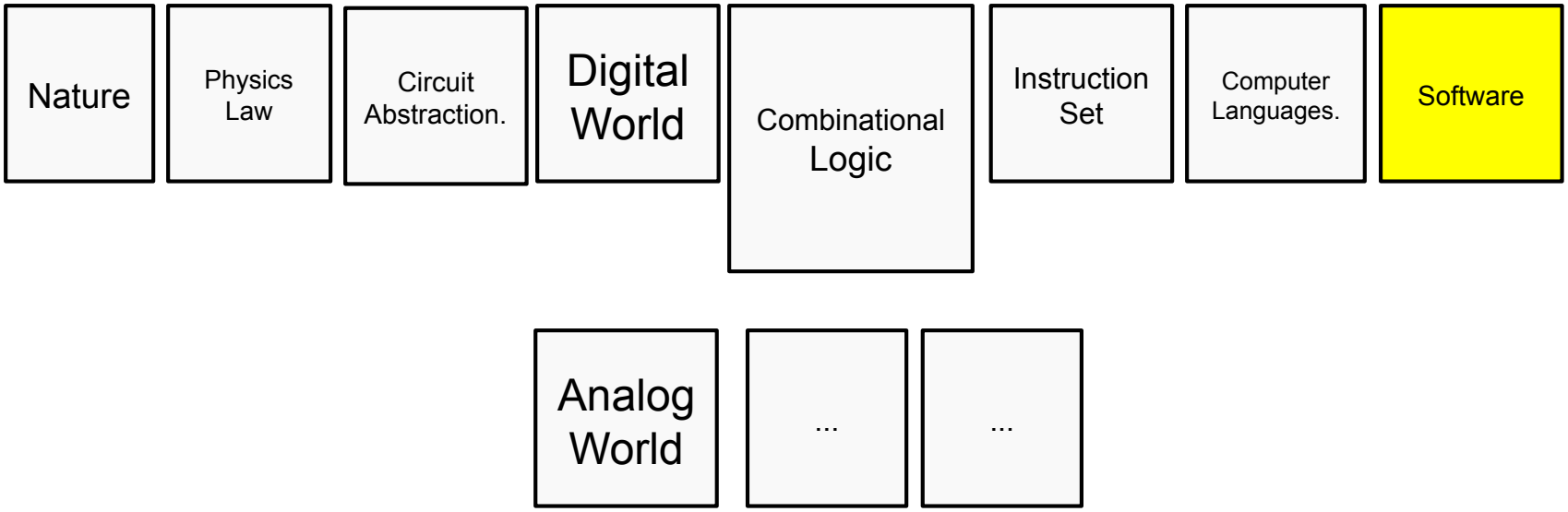
MACHINE INSTRUCTIONS (Logic 1 = Low Voltage = Negative Voltage; Logic 0 = High Voltage = Ground)

MNEMONIC	DESCRIPTION OF OPERATION	OPR $D_3\ D_2\ D_1\ D_0$	OPA $D_3\ D_2\ D_1\ D_0$
NOP	No operation.	0 0 0 0	0 0 0 0
*JCN	Jump to ROM address $A_2\ A_2\ A_2\ A_2$, $A_1\ A_1\ A_1\ A_1$ (within the same ROM that contains this JCN instruction) if condition $C_1\ C_2\ C_3\ C_4$ [1] is true, otherwise skip (go to the next instruction in sequence).	0 0 0 1 $A_2\ A_2\ A_2\ A_2$	$C_1\ C_2\ C_3\ C_4$ $A_1\ A_1\ A_1\ A_1$
*FIM	Fetch immediate (direct) from ROM Data D_2, D_1 to index register pair location RRR. [2]	0 0 1 0 $D_2\ D_2\ D_2\ D_2$	R R R 0 $D_1\ D_1\ D_1\ D_1$
SRC	Send register control. Send the address (contents of index register pair RRR) to ROM and RAM at X_2 and X_3 time in the Instruction Cycle.	0 0 1 0	R R R 1
FIN	Fetch indirect from ROM. Send contents of index register pair location 0 out as an address. Data fetched is placed into register pair location RRR.	0 0 1 1	R R R 0
JIN	Jump indirect. Send contents of register pair RRR out as an address at A_1 and A_2 time in the Instruction Cycle.	0 0 1 1	R R R 1
*JUN	Jump unconditional to ROM address A_3, A_2, A_1.	0 1 0 0 $A_2\ A_2\ A_2\ A_2$	$A_3\ A_3\ A_3\ A_3$ $A_1\ A_1\ A_1\ A_1$
*JMS	Jump to subroutine ROM address A_3, A_2, A_1, save old address. (Up 1 level in stack.)	0 1 0 1 $A_2\ A_2\ A_2\ A_2$	$A_3\ A_3\ A_3\ A_3$ $A_1\ A_1\ A_1\ A_1$
INC	Increment contents of register RRRR. [3]	0 1 1 0	R R R R
*ISZ	Increment contents of register RRRR. Go to ROM address A_2, A_1 (within the same ROM that contains this ISZ instruction) if result ≠ 0, otherwise skip (go to the next instruction in sequence).	0 1 1 1 $A_2\ A_2\ A_2\ A_2$	R R R R $A_1\ A_1\ A_1\ A_1$
ADD	Add contents of register RRRR to accumulator with carry.	1 0 0 0	R R R R
SUB	Subtract contents of register RRRR to accumulator with borrow.	1 0 0 1	R R R R
LD	Load contents of register RRRR to accumulator.	1 0 1 0	R R R R
XCH	Exchange contents of index register RRRR and accumulator.	1 0 1 1	R R R R
BBL	Branch back (down 1 level in stack) and load data DDDD to accumulator.	1 1 0 0	D D D D
LDM	Load data DDDD to accumulator.	1 1 0 1	D D D D

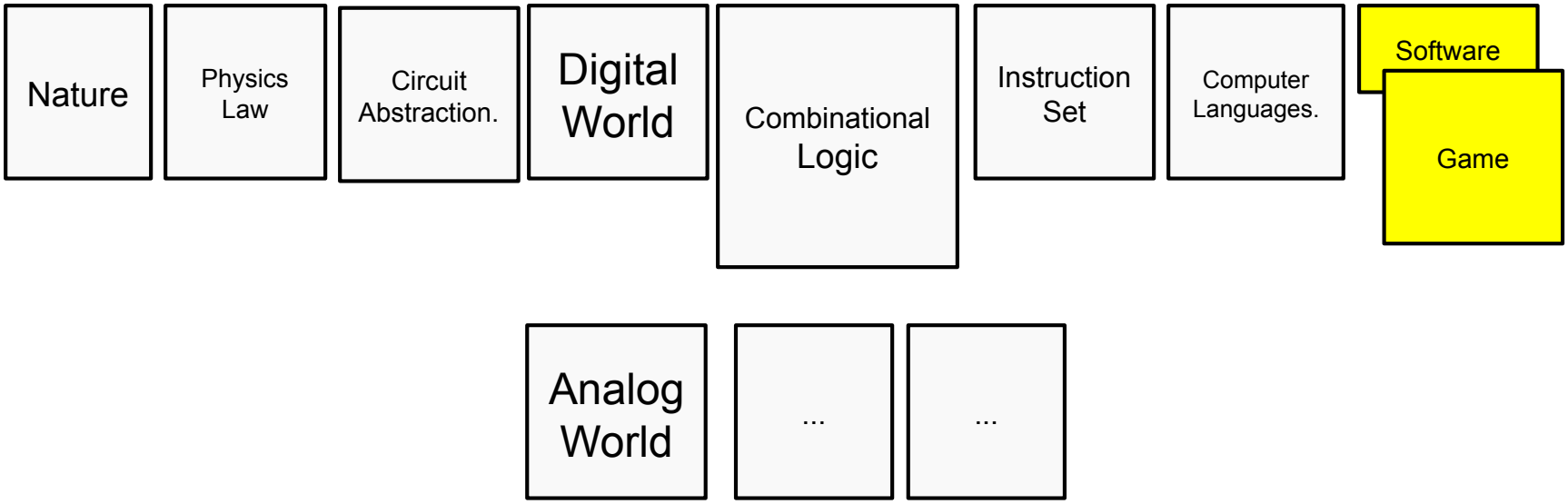
Computer Language

- Assembler is machine specific.
- What does it mean?
  - Have to write machine dependent assembler for machines.
  - Programmer is lazy! Don’t like this.
- So computer language has been invented.
  - C, C++, C#
  - Java
  - Basic and so on...

Where we are


Nature
Physics Law
Circuit Abstraction.
Digital World
Combinational Logic
Instruction Set
Computer Languages.
Software
Analog World
...
...

Computer game is a computer software.



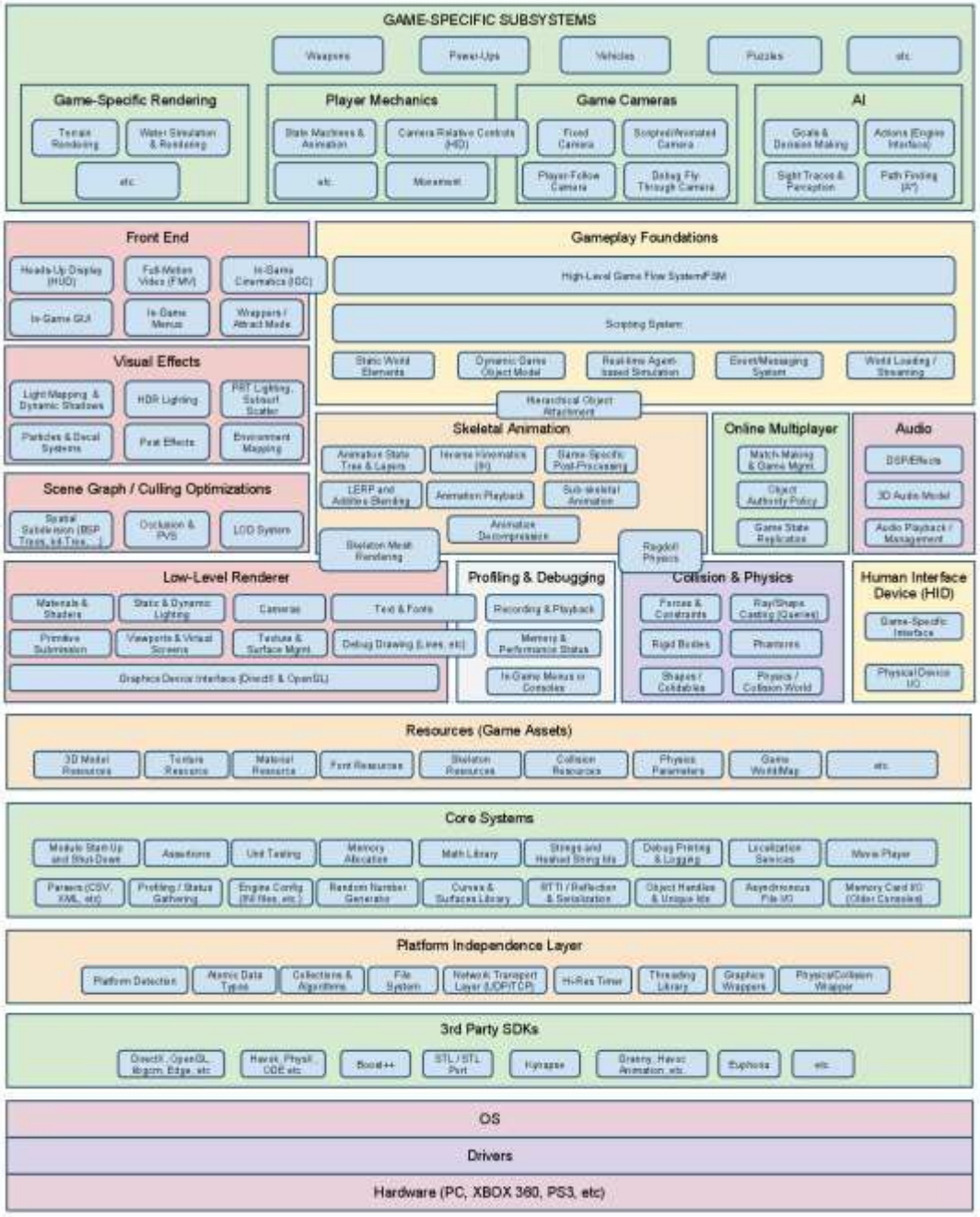
What is Game Engine?

- http:// en.wikipedia.org/wiki/Game_engine

- A **game engine** is a software framework designed for the creation and development of video games.

Game Engine

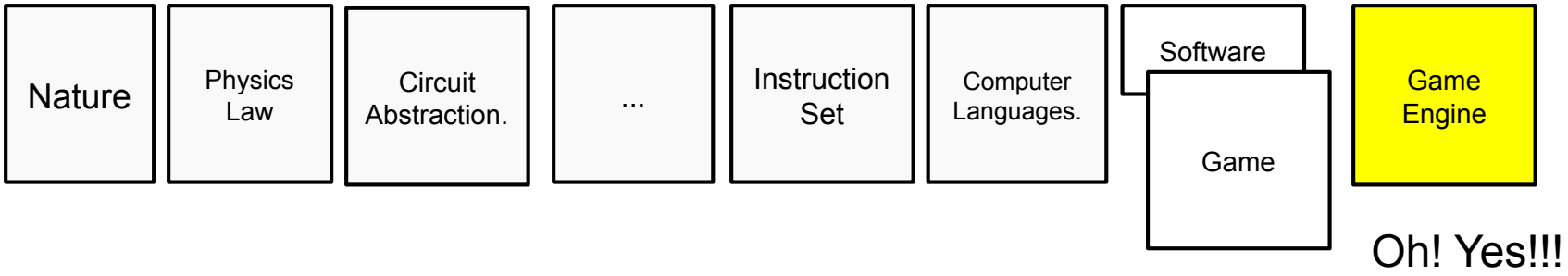
- Games are very complicated software.
- For example, Rendering, Physics, AI, Sounds, Input and so on.


GAME-SPECIFIC SUBSYSTEMS
Weapons
Power-Ups
Vehicles
Puzzles
etc.
Game-Specific Rendering
Terrain Rendering
Water Simulation & Rendering
etc.
Player Mechanics
State Machines & Animation
Camera-Relative Controls (HID)
etc.
Movement
Game Cameras
Fixed Camera
Scripted/Animated Camera
Player-Follow Camera
Debug Fly-Through Camera
AI
Goals & Decision Making
Actions (Engine Interface)
Sight Traces & Perception
Path Finding (A*)
Front End
Heads-Up Display (HUD)
Full-Motion Video (FMV)
In-Game Cinematics (IGC)
In-Game GUI
In-Game Menus
Wrappers / Attract Mode
Visual Effects
Light Mapping & Dynamic Shadows
HDR Lighting
PRT Lighting, Subsurf Scatter
Particles & Decal Systems
Post Effects
Environment Mapping
Scene Graph / Culling Optimizations
Spatial Subdivision (BSP Trees, kd-Trees, ...)
Occlusion & PVS
LOD System
Gameplay Foundations
High-Level Game Flow System/FSM
Scripting System
Static World Elements
Dynamic Game Object Model
Real-time Agent-based Simulation
Event/Messaging System
World Loading / Streaming
Hierarchical Object Attachment
Skeletal Animation
Animation State Tree & Layers
Inverse Kinematics (IK)
Game-Specific Post-Processing
LERP and Additive Blending
Animation Playback
Sub-skeletal Animation
Animation Decompression
Skeleton Mesh Rendering
Ragdoll Physics
Online Multiplayer
Match-Making & Game Mgmt
Object Authority Policy
Game State Replication
Audio
DSP/Effects
3D Audio Model
Audio Playback / Management
Low-Level Renderer
Materials & Shaders
Static & Dynamic Lighting
Cameras
Text & Fonts
Primitive Submission
Viewports & Virtual Screens
Texture & Surface Mgmt
Debug Drawing (Lines, etc)
Graphics Device Interface (Direct3 & OpenGL)
Profiling & Debugging
Recording & Playback
Memory & Performance Stats
In-Game Menus or Consoles
Collision & Physics
Forces & Constraints
Ray/Shape Casting (Queries)
Rigid Bodies
Phantoms
Shapes / Collidables
Physics / Collision World
Human Interface Device (HID)
Game-Specific Interface
Physical Device I/O
Resources (Game Assets)
3D Model Resources
Texture Resource
Material Resource
Font Resources
Skeleton Resources
Collision Resources
Physics Parameters
Game World/Map
etc.
Core Systems
Module Start-Up and Shut-Down
Assertions
Unit Testing
Memory Allocation
Math Library
Strings and Hashed String Ids
Debug Printing & Logging
Localization Services
Movie Player
Parsers (CSV, XML, etc)
Profiling / Stats Gathering
Engine Config (INI files, etc.)
Random Number Generator
Curves & Surfaces Library
RTTI / Reflection & Serialization
Object Handles & Unique Ids
Asynchronous File I/O
Memory Card I/O (Older Consoles)
Platform Independence Layer
Platform Detection
Atomic Data Types
Collections & Algorithms
File System
Network Transport Layer (UDP/TCP)
Hi-Res Timer
Threading Library
Graphics Wrappers
Physics/Collision Wrapper
3rd Party SDKs
DirectX, OpenGL, libgcm, Edge, etc.
Havok Physics, ODE etc.
Boost++
STL / STL Port
Kynapse
Granny, Havok Animation, etc.
Euphoria
etc.
OS
Drivers
Hardware (PC, XBOX 360, PS3, etc)

Do I need to implement all of those?!?!

- YES
  - 15 years ago
- NO
  - use abstraction = game engine

Game Engine



Oh! Yes!!!

Game Engine does every dirty/hard works for you.

- To construct every game systems from the start is hard.
- Game Engine **abstract** all of those hard works for gameplay programmer so programmer have more time to concentrate on gameplay instead of dirty/hard works.

What is Platform?

- http:// en.wikipedia.org/wiki/Computing_platform

- A **computing platform** is, in the most general sense, whatever pre-existing environment a piece of software is designed to run within, obeying its constraints, and making use of its facilities. Typical platforms include a hardware architecture , an operating system (OS), and runtime libraries.[1]

Meaning of Platform is very wide.

- For Game Engine, Platform is a machine environment.
- For Example
  - xbox360
  - PC
  - PS3
  - Web
  - Android
  - iOS

What is Cross-Platform?

- Cross-Platform
  - http:// en.wikipedia.org/wiki/Cross-platform
- In computing, **cross-platform**, or **multi-platform**, is an attribute conferred to computer software or computing methods and concepts that are implemented and inter-operate on multiple computer platforms.[1] The software and methods are also said to be **platform independent**. Cross-platform software may be divided into two types; one requires individual building or compilation for each platform that it supports, and the other one can be directly run on any platform without special preparation, e.g., software written in aninterpreted language or pre-compiled portable bytecode for which the interpreters or run-time packages are common or standard components of all platforms. [2]

Simply cross-platform software runs on different machines.

For example, Java application generate ‘byte code’ and it compiled on the fly based on the machines.

Cross-Platform Computer Software

- Normally program runs on specific machines such as PC, Mac and so on.
- If we want program runs on every machines(PC, Mac for example) programmer needs to make two programs.
- Some machine's environment are exactly same but most of cases it is not.
- What's mean?
  - Programmer need to make a program per machines.

Why are apps added first on Android and later on iOS or vice versa and not simultaneously? Is it the server cost which is a major factor? - Quora

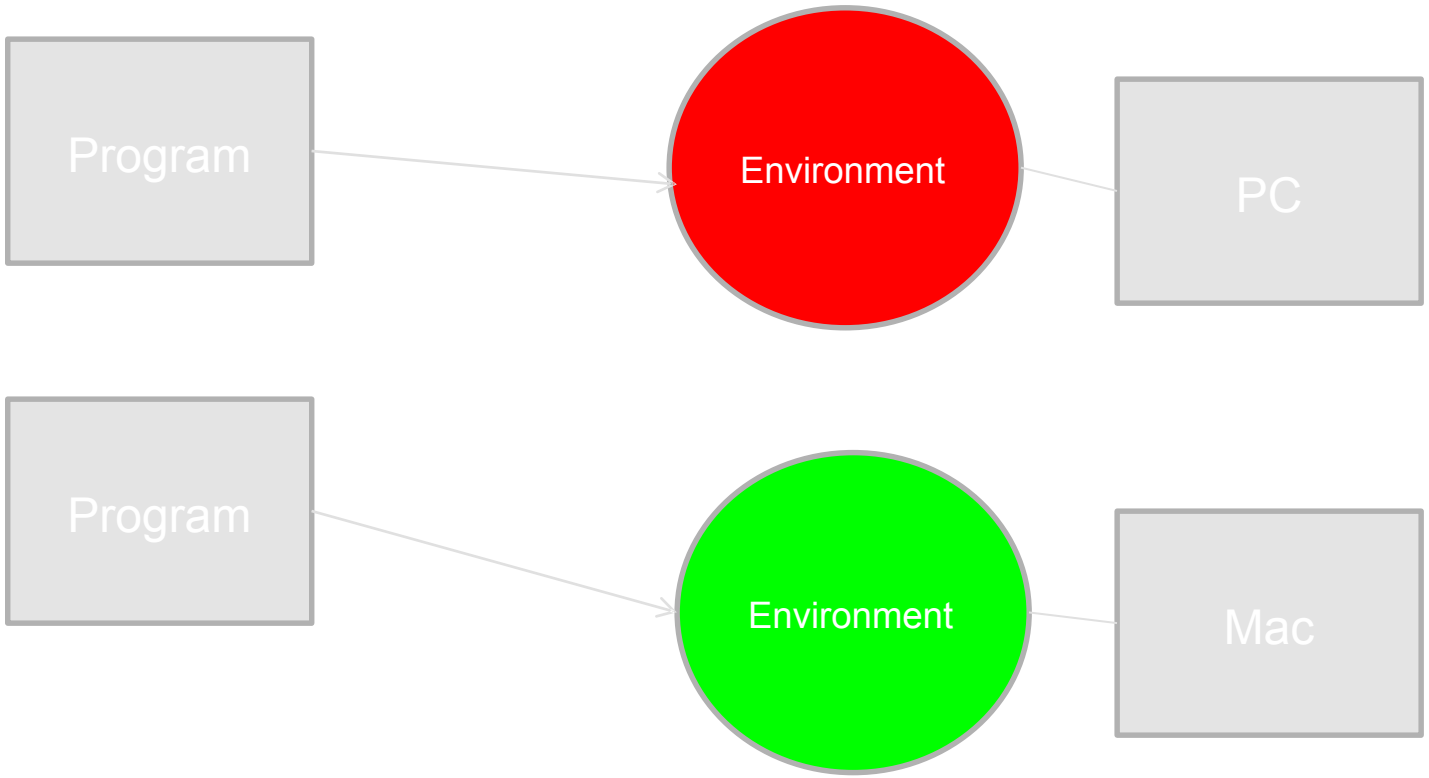
2013 Is Android First Really a Myth?

Android vs iOS: which App Should You Develop First

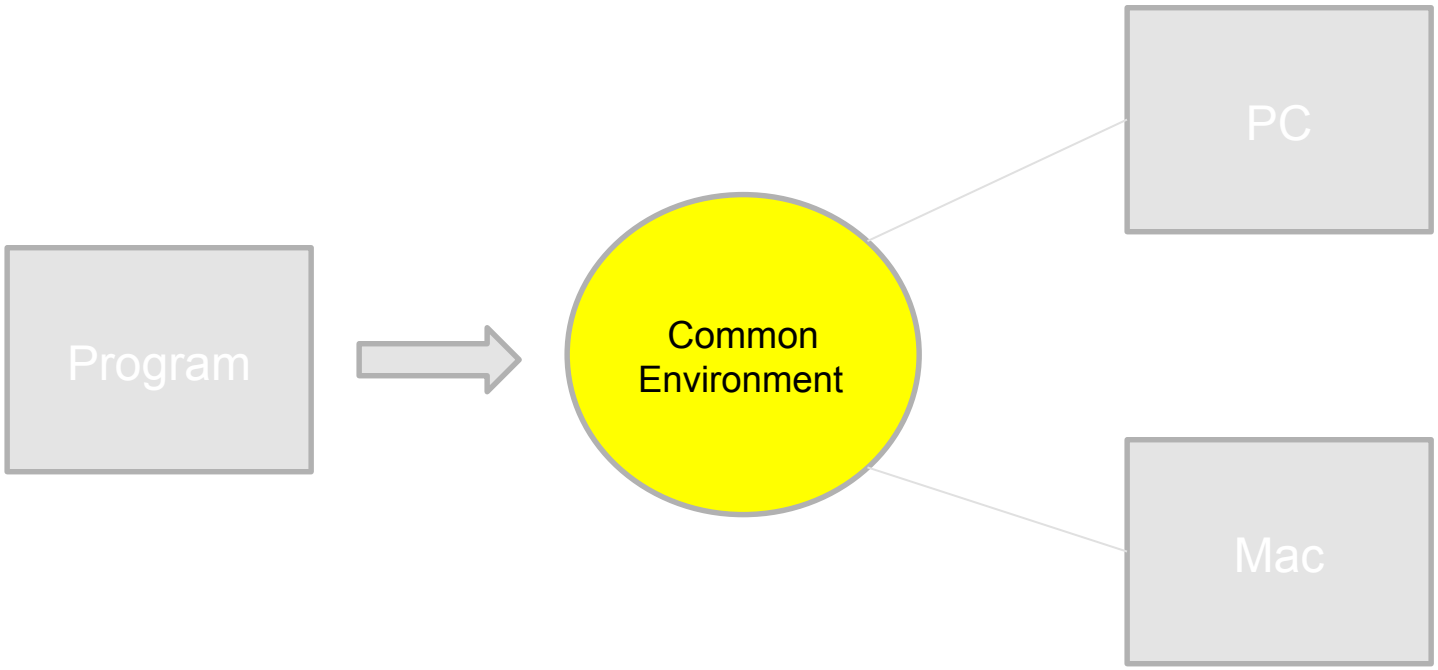
Why are the Angry Birds apps $0.99 on iOS but free on Android? - Quora

Concept of Cross-Platform Software

- Concept is simple



DRY (Don't repeat yourself)

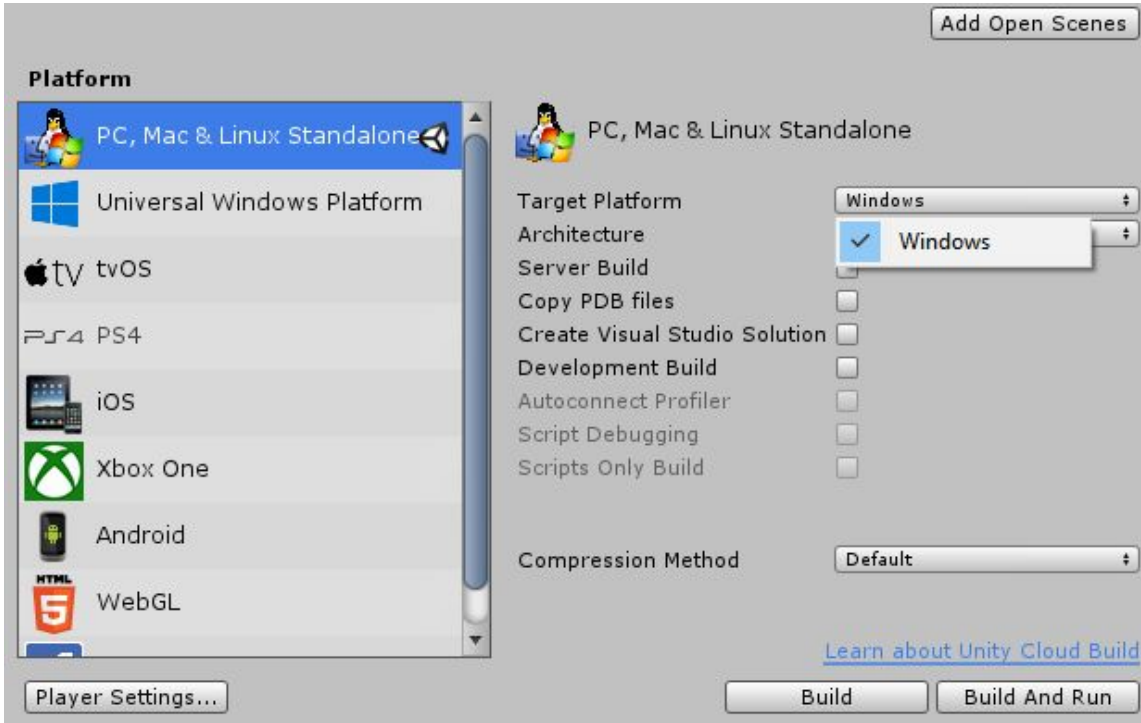


Example : Endian mode difference.

- PC uses little-endian mode.
- PS3(PowerPC) uses big-endian mode.
- What is a problem?
  - Tool and Game
- I choose big-endian mode
  - I don’t want to waste of time for game.
  - PC(For developer) version is ok to be slow.
- Support MemoryStream Class
  - Programmer don’t need to care about the file format.
  - Just use MemoryStream’s interface (read, write) and MemoryStream class does all dirty/hard works.

Case Study : Unity

- Unity is a cross-platform game engine.
- Unity is not source based game engine.
- Gameplay programmer can use C#, Boo, Java Script for gameplay logic.
- Support all kind of platforms such as pc, android, iOS, web, console and so on.



Case Study : Unity

- Good Things
  - It support many platforms and easy to change platforms.
- Fact
    - Many Korean game companies (include small studio) uses Unity Engine for their games.
- Bad Things
  - Performance issue can be raised.
  - With my experience native app is 6 times smaller than the app with Unity Engine.
  - More than 2 times slower than native app.
  - Source is not available (we can buy it if we have money)
    - Can’t do anything if there is a problem happen. Have to wait for next update.

Summary

- Computer game is equal to computer software.
- Abstraction is the best tool.
- Cross-platform is must have features of recent game engines.
- Game Engine has trade offs.
  - Compare to native program.
  - Normally native program is faster/smaller than the program use game engines.
  - If you are making an engine yourself you have to implement all of those features!

BONUS (“what about…”) Platform specific features

- Wii vs Kinect (vs Move)

  e.g. sports

- smartphone vs PC
  (3DS, Wii U, Switch)

  e.g. Rayman [Rayman Legends Wii U]

- Sony PS VR vs
  room scale VR (HTC, Valve) vs
  MS Hololens

List of video games that support cross-platform play

Cross-platform play

Random problems

tylko **ZAJAWKA**
- całość na
Gry Komputerowe
II stopień,
spec. PSI

[Apex Legends is out on the Switch, but it's missing a key featu… cross-progression]

[Lego Batman 2 - platform differences? : 3DS]

[Difference to Console version - LEGO Batman: The Videogame]

Aladdin - https://www.facebook.com/marekopel/posts/1015915314830972 5

Guitar Hero (DS) https://www.youtube.com/watch?v=BP_nOi0VrjI (Wii+DS) https://www.youtube.com/watch?v=KAZ715JtjZE

Minecraft Java vs Bedrock - [40 RÓŻNIC między MINECRAFT JAVA, a BEDROCK]

Fortnite case study

https://www.phonearena.com/news/fortnite-ios-android-console-ps4-xbox-pc-differences-comparison_id103461

Fortnite PC vs MOBILE - GAMEPLAY

Fortnite Graphic - PC vs Samsung Galaxy Note 9 (Android)

Backward_compatibility

https://en.wikipedia.org/wiki/Virtual_Console

https://en.wikipedia.org/wiki/Nintendo_Classics

https://en.wikipedia.org/wiki/Backward_compatibility#Costs PS3 > PS2